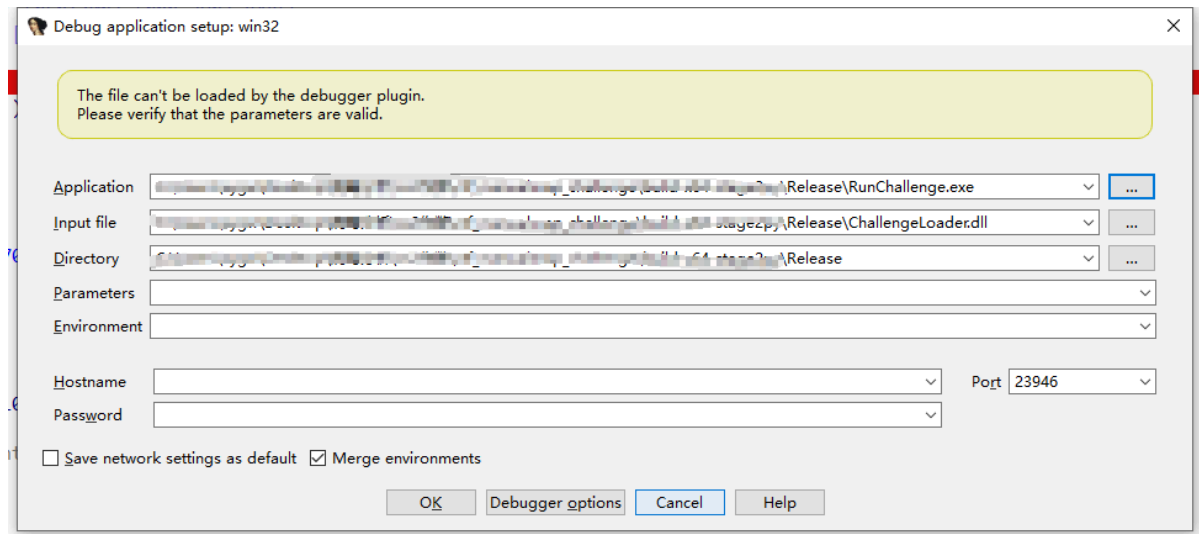


DLL的秘密 wp

动态调试

使用win10操作系统，自行编写加载DLL的程序，并调用导出函数start.



更改Application为你自行编写的程序即可

分析

注意到窗口过程回调函数

```
WndClass.lpfnWndProc = (WNDPROC)sub_7FFED4941A70;
```

注意到当回调函数获取到WM_COMMAND消息时，会调用一个函数

```
default:
    if ( Msg != 273 || (_WORD)wParam != 1002 || !WindowLongPtrW || !*(__QWORD *) (WindowLongPtrW + 8) )
        return DefWindowProcW(hWndParent, Msg, wParam, (LPARAM)lParam);
    sub_7FFED4942190(String, 0, 256);
    sub_7FFED4942190(Text, 0, 512);
    GetWindowTextW(*(HWND *)WindowLongPtrW, String, 128);
    v9 = (*(__int64 (__fastcall **)(WCHAR *, WCHAR *, __int64))(WindowLongPtrW + 8))(String, Text, 256);
    v10 = L"Access Check";
    v11 = 48;
    if ( v9 )
    {
        v11 = 64;
        v10 = L"MiniL 2026";
    }
    MessageBoxW(hWndParent, Text, v10, v11);
    break;
}
```

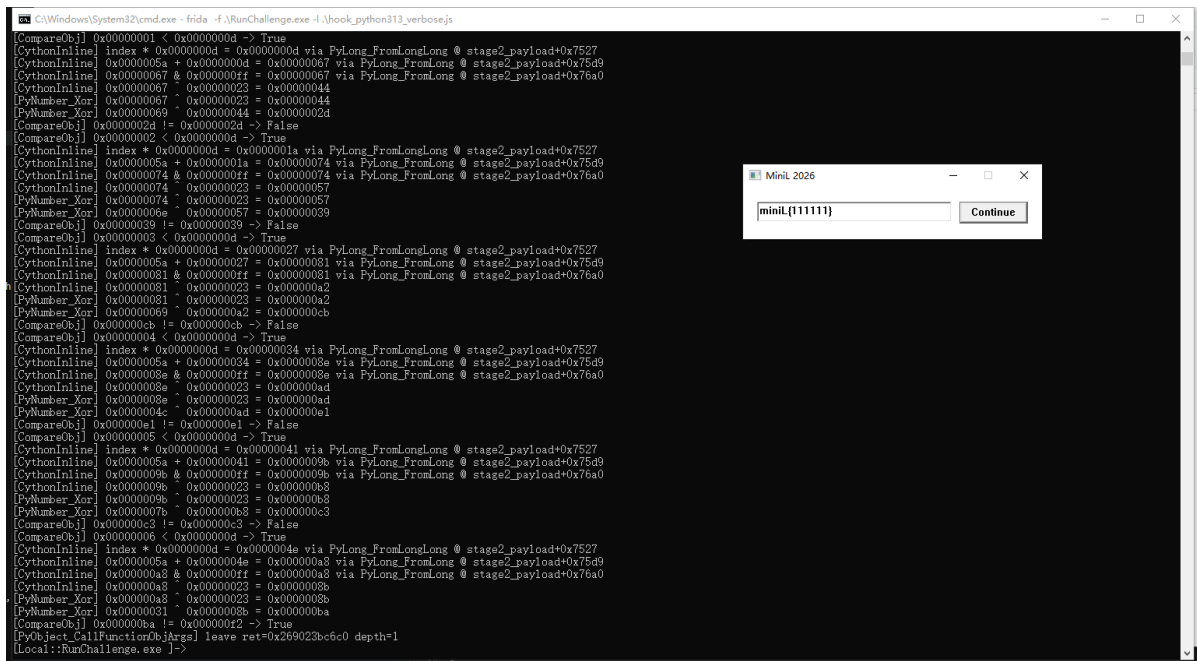
下断点，F7进入并继续调试

在ida中发现加载了

```
7FFF453A0000: loaded C:\Users\xygw\AppData\Local\Temp\MiniL2026_PythonRuntime\stage2_payload.pyd
```

HOOK

在处理逻辑运算时,cpython会调用一系列cpython api，如：PyNumber_And，PyNumber_Or，PyNumber_Xor等。如果我们能拦截这些api并获取操作数和结果，就能直接对执行流进行分析。



```
'use strict';
/**
 * =====
 * Frida hook script for Python3 numeric & compare ops
 * - Hooks PyNumber_* (add/sub/mul/div, bit ops, modulo, shifts, power)
 * - Hooks PyObject_RichCompareBool / PyObject_RichCompare
 * - Hooks PyLong_FromString / PyLong_FromUnicodeObject
 * - Fixed type error: use int64/uint64 instead of longlong
 * - Added fixed-width HEX output (e.g. 0x12345678)
 * - Adapted for python313.dll loaded at runtime
 * =====
 */

const logFilePath = './log.txt';
const logFile = new File(logFilePath, 'w');
const TRACE_ONLY_DURING_PY_CALL = true;
const TRACE_CYTHON_INLINE_INTS = true;
const SUPPRESS_TYPEERROR_OUTPUT = true;

let pymod = null;
let symbols = null;
let hooksInstalled = false;
let traceDepth = 0;
let stage2PayloadModule = null;
let pendingInlineInts = [];
let lastLoopIndex = null;

let PyUnicode_AsUTF8 = null;
let PyObject_Repr = null;
let PyObject_Str = null;
let PyLong_Check = null;
let PyLong_AsLongLong = null;
let PyLong_AsUnsignedLongLongMask = null;
let PyLong_Type = null;
let Py_DecRef = null;
let PyErr_Fetch = null;
let PyErr_Restore = null;
```

```

function logLine(s) {
    if (SUPPRESS_TYPEERROR_OUTPUT && /TypeError/i.test(String(s))) {
        return;
    }
    logFile.write(s + '\n');
    logFile.flush();
    console.log(s);
}

function shouldTrace() {
    return !TRACE_ONLY_DURING_PY_CALL || traceDepth > 0;
}

function findStage2PayloadModule() {
    if (stage2PayloadModule) return stage2PayloadModule;

    const mods = enumerateModulesCompat();
    for (const m of mods) {
        const n = m.name.toLowerCase();
        if (n === 'stage2_payload.pyd' || n.indexOf('stage2_payload') !== -1) {
            stage2PayloadModule = m;
            return stage2PayloadModule;
        }
    }

    return null;
}

function pointerInModule(ptrValue, mod) {
    if (!ptrValue || !mod) return false;

    try {
        const base = mod.base;
        const end = mod.base.add(mod.size);
        return ptrValue.compare(base) >= 0 && ptrValue.compare(end) < 0;
    } catch (_) {
        return false;
    }
}

function moduleOffset(ptrValue, mod) {
    if (!ptrValue || !mod) return '<unknown>';

    try {
        return ptrValue.sub(mod.base).toString();
    } catch (_) {
        return '<unknown>';
    }
}

// -----
// Adaptation: support Frida 17 API variants and runtime-loaded python DLL.

function enumerateModulesCompat() {
    if (typeof Process.enumerateModulesSync === 'function') {

```

```

        return Process.enumerateModulesSync();
    }
    return Process.enumerateModules();
}

function findModuleCompat(name) {
    if (typeof Process.findModuleByName === 'function') {
        return Process.findModuleByName(name);
    }
    try {
        return Process.getModuleByName(name);
    } catch (_) {
        return null;
    }
}

function findExportCompat(moduleName, exportName) {
    try {
        if (typeof Module.getExportByName === 'function') {
            return Module.getExportByName(moduleName, exportName);
        }
    } catch (_) {
    }

    try {
        if (typeof Module.findExportByName === 'function') {
            const addr = Module.findExportByName(moduleName, exportName);
            if (addr) return addr;
        }
    } catch (_) {
    }

    const mod = findModuleCompat(moduleName);
    if (!mod) return null;

    try {
        if (typeof mod.getExportByName === 'function') {
            return mod.getExportByName(exportName);
        }
    } catch (_) {
    }

    try {
        if (typeof mod.findExportByName === 'function') {
            const addr = mod.findExportByName(exportName);
            if (addr) return addr;
        }
    } catch (_) {
    }

    try {
        if (typeof mod.enumerateExports === 'function') {
            for (const item of mod.enumerateExports()) {
                if (item.name === exportName) return item.address;
            }
        }
    }
}

```

```

    } catch (_) {
    }

    try {
        if (typeof Module.getGlobalExportByName === 'function') {
            return Module.getGlobalExportByName(exportName);
        }
    } catch (_) {
    }

    return null;
}

function readUtf16Safe(p) {
    try {
        if (!p || p.isNull()) return '<null>';
        return p.readUtf16String();
    } catch (_) {
        return '<bad-wstr>';
    }
}

function findPythonModule() {
    const mods = enumerateModulesCompat();
    for (const m of mods) {
        const n = m.name.toLowerCase();
        if (n.startsWith('python3') && n.endsWith('.dll')) return m;
        if (n.startsWith('python') && n.endsWith('.dll')) return m;
    }
    return null;
}

// safe wrapper to resolve exports and print status
function exp(name) {
    try {
        const addr = findExportCompat(pymod.name, name);
        if (addr) {
            logLine(`[+] export ${name} => ${addr}`);
            return addr;
        }
        logLine(`[-] export ${name} not found`);
        return null;
    } catch (e) {
        logLine(`[-] export ${name} not found`);
        return null;
    }
}

// -----

function tryCreateNativeFunction(addr, retType, argTypes, friendlyName) {
    if (!addr) return null;
    try {
        const nf = new NativeFunction(addr, retType, argTypes);
        logLine(`[+] NativeFunction created for ${friendlyName}`);
        return nf;
    }

```

```

    } catch (e) {
        logLine(`[-] Failed creating NativeFunction for ${friendlyName}:
${e.message}`);
        return null;
    }
}

// -----

function toHexFixed(num, width = 8) {
    try {
        let big;
        if (typeof num === 'bigint') big = num;
        else if (typeof num === 'number') big = BigInt(num >>> 0);
        else if (typeof num === 'string') {
            if (/^0x[0-9a-fA-F]+$/.test(num)) big = BigInt(num);
            else if (/^-?\d+$/.test(num)) {
                big = BigInt(num);
                if (big < 0n) {
                    const bits = BigInt(width * 4);
                    big = (big & ((1n << bits) - 1n));
                }
            } else return num;
        } else {
            const text = String(num);
            if (/^0x[0-9a-fA-F]+$/.test(text)) big = BigInt(text);
            else if (/^-?\d+$/.test(text)) big = BigInt(text);
            else return `

```

```

function firstIntegerArg(context, fallbackArg) {
    if (Process.arch === 'x64' && context && context.rcx) {
        return context.rcx;
    }
    return fallbackArg;
}

function hexToBigIntOrNull(text) {
    try {
        if (typeof text !== 'string') return null;
        if (!/^0x[0-9a-fA-F]+$/.test(text)) return null;
        return BigInt(text);
    } catch (_) {
        return null;
    }
}

function subtractHexToFixed(lhs, rhs, width = 8) {
    const left = hexToBigIntOrNull(lhs);
    if (left === null) return '<unknown>';
    return toHexFixed((left - BigInt(rhs)).toString(), width);
}

function multiplyHexToFixed(lhs, rhs, width = 8) {
    const left = hexToBigIntOrNull(lhs);
    if (left === null) return '<unknown>';
    return toHexFixed((left * BigInt(rhs)).toString(), width);
}

function addHexToFixed(lhs, rhs, width = 8) {
    const left = hexToBigIntOrNull(lhs);
    if (left === null) return '<unknown>';
    return toHexFixed((left + BigInt(rhs)).toString(), width);
}

function andHexToFixed(lhs, rhs, width = 8) {
    const left = hexToBigIntOrNull(lhs);
    if (left === null) return '<unknown>';
    return toHexFixed((left & BigInt(rhs)).toString(), width);
}

function isDecimalString(s) {
    return typeof s === 'string' && /^-?\d+$/.test(s);
}

function isHexString(s) {
    return typeof s === 'string' && /^0x[0-9a-fA-F]+$/.test(s);
}

function formatForLogging(s, width = 8) {
    if (s === undefined || s === null) return s;
    if (isHexString(s)) return s;
    if (isDecimalString(s)) return toHexFixed(s, width);
    const m = /(-?\d+)$/.exec(String(s).trim());
    if (m) return toHexFixed(m[1], width);
    return s;
}

```

```

function isHexValue(s, expected) {
    try {
        if (typeof s !== 'string') return false;
        if (!/^0x[0-9a-fA-F]+$/.test(s)) return false;
        return BigInt(s) === BigInt(expected);
    } catch (_) {
        return false;
    }
}

function isFormattedHex(s) {
    return typeof s === 'string' && /^0x[0-9a-fA-F]+$/.test(s);
}

function safeUtf8FromPyStr(pyStrPtr) {
    if (!pyStrPtr || pyStrPtr.isNull()) return '<utf8 NULL>';
    try {
        if (!PyUnicode_AsUTF8) return '<no PyUnicode_AsUTF8>';
        const cstr = PyUnicode_AsUTF8(pyStrPtr);
        if (cstr.isNull()) return '<utf8 NULL>';
        return cstr.readUtf8String();
    } catch {
        return '<utf8 read error>';
    }
}

function decRef(pyObjPtr) {
    if (!Py_DecRef || !pyObjPtr || pyObjPtr.isNull()) return;
    try {
        Py_DecRef(pyObjPtr);
    } catch (_) {
    }
}

function readPointersSlot(slot) {
    try {
        return slot.readPointer();
    } catch (_) {
        return ptr('0');
    }
}

function decRefIfSet(pyObjPtr) {
    if (pyObjPtr && !pyObjPtr.isNull()) {
        decRef(pyObjPtr);
    }
}

function withSavedPyErr(fn) {
    if (!PyErr_Fetch || !PyErr_Restore) {
        return fn();
    }

    const savedTypesSlot = Memory.alloc(Process.pointerSize);
    const savedValuesSlot = Memory.alloc(Process.pointerSize);

```



```

const savedTracesSlot = Memory.alloc(Process.pointerSize);
const currentTypesSlot = Memory.alloc(Process.pointerSize);
const currentValuesSlot = Memory.alloc(Process.pointerSize);
const currentTracesSlot = Memory.alloc(Process.pointerSize);

PyErr_Fetch(savedTypesSlot, savedValuesSlot, savedTracesSlot);

let result;
let thrown = null;
try {
    result = fn();
} catch (e) {
    thrown = e;
} finally {
    PyErr_Fetch(currentTypesSlot, currentValuesSlot, currentTracesSlot);
    decRefIfSet(readPointerSlot(currentTypesSlot));
    decRefIfSet(readPointerSlot(currentValuesSlot));
    decRefIfSet(readPointerSlot(currentTracesSlot));

    PyErr_Restore(
        readPointerSlot(savedTypesSlot),
        readPointerSlot(savedValuesSlot),
        readPointerSlot(savedTracesSlot)
    );
}

if (thrown) {
    throw thrown;
}
return result;
}

function getObjectType(pyObjPtr) {
    try {
        if (!pyObjPtr || pyObjPtr.isNull()) return null;
        return pyObjPtr.add(Process.pointerSize).readPointer();
    } catch (_) {
        return null;
    }
}

function isPyLongObject(pyObjPtr) {
    try {
        if (PyLong_Check && PyLong_Check(pyObjPtr) === 1) return true;
        if (!PyLong_Type) return false;
        const objType = getObjectType(pyObjPtr);
        return objType !== null && objType.equals(PyLong_Type);
    } catch (_) {
        return false;
    }
}

function objToStr(pyObjPtr) {
    if (!pyObjPtr || pyObjPtr.isNull()) return '<NULL>';
    try {
        return withSavedPyErr(() => {

```

```

    if (isPyLongObject(pyObjPtr)) {
        if (PyLong_AsUnsignedLongLongMask) {
            const u = PyLong_AsUnsignedLongLongMask(pyObjPtr);
            return toHexFixed(u);
        }
        if (PyLong_AsLongLong) {
            const v = PyLong_AsLongLong(pyObjPtr);
            return toHexFixed(v);
        }
    }
    if (PyObject_Str) {
        const sObj = PyObject_Str(pyObjPtr);
        if (sObj && !sObj.isNull()) {
            const text = safeUtf8FromPyStr(sObj);
            decRef(sObj);
            return text;
        }
    }
    if (PyObject_Repr) {
        const r = PyObject_Repr(pyObjPtr);
        if (r && !r.isNull()) {
            const text = safeUtf8FromPyStr(r);
            decRef(r);
            return text;
        }
    }
    return '<unrepresentable>';
});
} catch (e) {
    return '<repr error>';
}
}

function rememberInlineInt(factoryName, rawValue, returnAddress) {
    if (!TRACE_CYTHON_INLINE_INTS || !shouldTrace()) return;
    if (factoryName === 'PyLong_FromSsize_t') return;

    const mod = findStage2PayloadModule();
    if (!pointerInModule(returnAddress, mod)) return;

    const formatted = toHexFixed(String(rawValue), 8);
    pendingInlineInts.push({
        factory: factoryName,
        value: formatted,
        raw: String(rawValue),
        caller: moduleOffset(returnAddress, mod)
    });

    if (pendingInlineInts.length > 16) {
        pendingInlineInts.shift();
    }
}

function flushMaskInlineInts(xorLeft, xorRight, xorOutput) {
    if (!TRACE_CYTHON_INLINE_INTS) return;
    if (!isHexValue(xorRight, 0x23n) && !isHexValue(xorLeft, 0x23n)) return;

```

```

    if (pendingInlineInts.length < 2 && !lastLoopIndex) return;

    const steps = pendingInlineInts.slice(-3);
    let multiplyStep;
    let addStep;
    let andStep;

    if (steps.length >= 3 &&
        isFormattedHex(steps[0].value) &&
        isFormattedHex(steps[1].value) &&
        isFormattedHex(steps[2].value)) {
        multiplyStep = steps[0];
        addStep = steps[1];
        andStep = steps[2];
    } else if (steps.length >= 2 &&
        isFormattedHex(steps[0].value) &&
        isFormattedHex(steps[1].value)) {
        addStep = steps[0];
        andStep = steps[1];
        multiplyStep = {
            factory: 'inferred',
            value: subtractHexToFixed(addStep.value, 0x5an),
            caller: 'no-new-PyLong'
        };
    } else if (lastLoopIndex) {
        const mul = multiplyHexToFixed(lastLoopIndex, 0x0dn);
        const add = addHexToFixed(mul, 0x5an);
        const anded = andHexToFixed(add, 0xffn);
        multiplyStep = {factory: 'reconstructed-from-index', value: mul, caller:
'index-compare'};
        addStep = {factory: 'reconstructed-from-index', value: add, caller:
'index-compare'};
        andStep = {factory: 'reconstructed-from-index', value: anded, caller:
'index-compare'};
    } else {
        return;
    }

    logLine(`[CythonInline] index * 0x0000000d = ${multiplyStep.value} via
${multiplyStep.factory} @ stage2_payload+${multiplyStep.caller}`);
    logLine(`[CythonInline] 0x0000005a + ${multiplyStep.value} = ${addStep.value}
via ${addStep.factory} @ stage2_payload+${addStep.caller}`);
    logLine(`[CythonInline] ${addStep.value} & 0x000000ff = ${andStep.value} via
${andStep.factory} @ stage2_payload+${andStep.caller}`);
    logLine(`[CythonInline] ${andStep.value} ^ 0x00000023 = ${xorOutput}`);
    pendingInlineInts = [];
}

// -----

function hookLongConverters() {
    // Hook PyLong_FromUnicodeObject / PyLong_FromString
    if (symbols.PyLong_FromUnicodeObject) {
        Interceptor.attach(symbols.PyLong_FromUnicodeObject, {
            onEnter(args) {
                if (!shouldTrace()) {

```

```

        this.skip = true;
        return;
    }
    this.sRaw = objToStr(args[0]);
    this.base = args[1].toInt32();
},
onLeave(retval) {
    if (this.skip) return;
    const got = objToStr(retval);
    logLine(`[PyLong_FromUnicodeObject] int(s, base=${this.base})
s=${this.sRaw} -> ${formatForLogging(got)}`);
}
});
}

if (symbols.PyLong_FromString) {
    Interceptor.attach(symbols.PyLong_FromString, {
        onEnter(args) {
            if (!shouldTrace()) {
                this.skip = true;
                return;
            }
            try { this.s = args[0].readUtf8String(); } catch (e) { this.s =
'<read error>'; }
            this.base = args[2].toInt32();
        },
        onLeave(retval) {
            if (this.skip) return;
            const got = objToStr(retval);
            logLine(`[PyLong_FromString] int(s, base=${this.base})
s=${this.s} -> ${formatForLogging(got)}`);
        }
    });
}

function symbolFor(name) {
    const map = {
        PyNumber_Add: '+',
        PyNumber_InPlaceAdd: '+=',
        PyNumber_Subtract: '-',
        PyNumber_Multiply: '*',
        PyNumber_InPlaceMultiply: '*= ',
        PyNumber_And: '&',
        PyNumber_InPlaceAnd: '&= ',
        PyNumber_Or: '|',
        PyNumber_InPlaceOr: '|= ',
        PyNumber_Xor: '^',
        PyNumber_InPlaceXor: '^= ',
        PyNumber_Lshift: '<<',
        PyNumber_Rshift: '>>',
        PyNumber_InPlaceRshift: '>>= ',
        PyNumber_Power: '**',
        PyNumber_Remainder: '%',
        PyNumber_Index: 'index'
    };
};

```

```

    return map[name] || '?';
}

// -----
//
// function hookPyNumber(name, ptr) {
//     if (!ptr) return;
//     Interceptor.attach(ptr, {
//         onEnter(args) {
//             this.aPtr = args[0];
//             this.bPtr = args[1];
//             this.aRaw = objToStr(this.aPtr);
//             this.bRaw = objToStr(this.bPtr);
//             this.a = formatForLogging(this.aRaw, 8);
//             this.b = formatForLogging(this.bRaw, 8);
//             try {
//                 this.cPtr = args[2];
//                 if (this.cPtr && !this.cPtr.isNull()) {
//                     this.cRaw = objToStr(this.cPtr);
//                     this.c = formatForLogging(this.cRaw, 8);
//                 } else {
//                     this.cPtr = null;
//                 }
//             } catch (e) { this.cPtr = null; }
//         },
//         onLeave(retval) {
//             if (name === 'PyNumber_Power') {
//                 if (this.cPtr) {
//                     logLine(`[${name}] pow(${this.a}, ${this.b}, ${this.c}) =
${formatForLogging(objToStr(retval), 8)}`);
//                 } else {
//                     logLine(`[${name}] ${this.a} ** ${this.b} =
<suppressed>`);
//                 }
//                 return;
//             }
//             logLine(`[${name}] ${this.a} ${symbolFor(name)} ${this.b} =
${formatForLogging(objToStr(retval), 8)}`);
//         }
//     });
// }

function hookPyNumber(name, ptr) {
    if (!ptr) return;
    Interceptor.attach(ptr, {
        onEnter(args) {
            if (!shouldTrace()) {
                this.skip = true;
                return;
            }
            // onEnter logic is unchanged: capture raw pointers and strings.
            this.aPtr = args[0];
            this.aRaw = objToStr(this.aPtr);
            this.a = formatForLogging(this.aRaw, 8);

            if (name === 'PyNumber_Index') {

```

```

        this.bPtr = null;
        this.bRaw = null;
        this.b = null;
        this.cPtr = null;
        this.cRaw = null;
        this.c = null;
    } else {
        this.bPtr = args[1];
        this.bRaw = objToStr(this.bPtr);
        this.b = formatForLogging(this.bRaw, 8);
    }

    try {
        this.cPtr = args[2];
        if (name === 'PyNumber_Power' && this.cPtr &&
!this.cPtr.isNull()) {
            this.cRaw = objToStr(this.cPtr);
            this.c = formatForLogging(this.cRaw, 8);
        } else {
            this.cPtr = null;
            this.cRaw = null;
            this.c = null;
        }
    } catch (e) {
        this.cPtr = null;
        this.cRaw = null;
        this.c = null;
    }
},
onLeave(retval) {
    if (this.skip) return;
    let aStr = this.aRaw.length > 32 ? 'num' : this.a;
    let bStr = this.bRaw && this.bRaw.length > 32 ? 'num' : this.b;
    const outRaw = objToStr(retval);
    let outFmt = outRaw.length > 32 ? 'num' : formatForLogging(outRaw,
8);

    if (name === 'PyNumber_Index') {
        logLine(`[${name}] index(${aStr}) = ${outFmt}`);
        return;
    }

    if (name === 'PyNumber_Xor') {
        flushMaskInlineInts(aStr, bStr, outFmt);
    }

    if (name === 'PyNumber_Power') {
        let modStr = this.c ? (this.cRaw.length > 32 ? 'num' : this.c) :
'None';
        logLine(`[${name}] pow(${aStr}, ${bStr}, ${modStr}) =
${outFmt}`);
        return;
    }

    logLine(`[${name}] ${aStr} ${symbolFor(name)} ${bStr} = ${outFmt}`);
}

```

```

    });
}
// -----

function hookNumberOps() {
    [
        'PyNumber_Add',
        'PyNumber_InPlaceAdd',
        'PyNumber_Subtract',
        'PyNumber_Multiply',
        'PyNumber_InPlaceMultiply',
        'PyNumber_And',
        'PyNumber_InPlaceAnd',
        'PyNumber_Or',
        'PyNumber_InPlaceOr',
        'PyNumber_Xor',
        'PyNumber_InPlaceXor',
        'PyNumber_Lshift',
        'PyNumber_Rshift',
        'PyNumber_InPlaceRshift',
        'PyNumber_Remainder',
        'PyNumber_Index',
        'PyNumber_Power'
    ].forEach(n => hookPyNumber(n, symbols[n]));
}

function hookCompareOps() {
    const cmpOps = ['<', '<=', '==', '!=', '>', '>='];

    if (symbols.PyObject_RichCompareBool) {
        Interceptor.attach(symbols.PyObject_RichCompareBool, {
            onEnter(args) {
                if (!shouldTrace()) {
                    this.skip = true;
                    return;
                }
                this.ptrA = args[0];
                this.ptrB = args[1];
                this.op = args[2].toInt32();
                this.aRaw = objToStr(this.ptrA);
                this.bRaw = objToStr(this.ptrB);
                this.a = formatForLogging(this.aRaw, 8);
                this.b = formatForLogging(this.bRaw, 8);
            },
            onLeave(retval) {
                if (this.skip) return;
                const opStr = cmpOps[this.op] || `op(${this.op})`;
                if (this.ptrA.equals(this.ptrB)) return;
                if (this.aRaw === this.bRaw && this.op === 2) return;
                if (opStr === '<' && isFormattedHex(this.a)) {
                    lastLoopIndex = this.a;
                }
                logLine(`[CompareBool] ${this.a} ${opStr} ${this.b} ->
${retval.toInt32()}`);
            }
        });
    }
}

```

```

}

if (symbols.PyObject_RichCompare) {
  Interceptor.attach(symbols.PyObject_RichCompare, {
    onEnter(args) {
      if (!shouldTrace()) {
        this.skip = true;
        return;
      }
      this.aRaw = objToStr(args[0]);
      this.bRaw = objToStr(args[1]);
      this.op = args[2].toInt32();
      this.a = formatForLogging(this.aRaw, 8);
      this.b = formatForLogging(this.bRaw, 8);
    },
    onLeave(retval) {
      if (this.skip) return;
      const opStr = cmpOps[this.op] || `op(${this.op})`;
      if (opStr === '<' && isFormattedHex(this.a)) {
        lastLoopIndex = this.a;
      }
      logLine(`[CompareObj] ${this.a} ${opStr} ${this.b} ->
${objToStr(retval)}`);
    }
  });
}

function hookCythonInlineIntFactories() {
  [
    ['PyLong_FromLong', 'int32'],
    ['PyLong_FromLongLong', 'int64'],
    ['PyLong_FromSsize_t', 'ssize']
  ].forEach(item => {
    const name = item[0];
    const kind = item[1];
    const ptr = symbols[name];
    if (!ptr) return;

    Interceptor.attach(ptr, {
      onEnter(args) {
        if (!shouldTrace()) {
          this.skip = true;
          return;
        }

        this.retaddr = this.returnAddress;
        const valueArg = firstIntegerArg(this.context, args[0]);
        this.value = nativeArgToSignedDecimal(valueArg, kind === 'int32'
? 32 : 64);
      },
      onLeave(_) {
        if (this.skip) return;
        rememberInlineInt(name, this.value, this.retaddr);
      }
    });
  });
}

```



```

});
}

function hookCallFunctionObjArgs() {
    if (!symbols(PyObject_CallFunctionObjArgs) return;

    Interceptor.attach(symbols(PyObject_CallFunctionObjArgs), {
        onEnter(args) {
            this.callable = args[0];
            this.arg0 = args[1];
            traceDepth++;
            logLine(`[PyObject_CallFunctionObjArgs] enter
callable=${this.callable} arg0=${this.arg0} depth=${traceDepth}`);
        },
        onLeave(retval) {
            logLine(`[PyObject_CallFunctionObjArgs] leave ret=${retval}
depth=${traceDepth}`);
            if (traceDepth > 0) traceDepth--;
        }
    });
}

function installHooks() {
    if (hooksInstalled) return;

    pymod = findPythonModule();
    if (!pymod) return;

    hooksInstalled = true;
    logLine(' using python module: ' + pymod.name + ' ' + pymod.base);

    // list of symbols
    symbols = {
        PyUnicode_AsUTF8: exp('PyUnicode_AsUTF8'),
        PyObject_Repr: exp('PyObject_Repr'),
        PyObject_Str: exp('PyObject_Str'),
        PyLong_Check: exp('PyLong_Check'),
        PyLong_AsLongLong: exp('PyLong_AsLongLong'),
        PyLong_AsUnsignedLongLongMask: exp('PyLong_AsUnsignedLongLongMask'),
        PyLong_Type: exp('PyLong_Type'),
        Py_DecRef: exp('Py_DecRef'),
        PyErr_Fetch: exp('PyErr_Fetch'),
        PyErr_Restore: exp('PyErr_Restore'),
        PyLong_FromLong: exp('PyLong_FromLong'),
        PyLong_FromLongLong: exp('PyLong_FromLongLong'),
        PyLong_FromSsize_t: exp('PyLong_FromSsize_t'),
        PyLong_FromString: exp('PyLong_FromString'),
        PyLong_FromUnicodeObject: exp('PyLong_FromUnicodeObject'),
        PyNumber_Add: exp('PyNumber_Add'),
        PyNumber_InPlaceAdd: exp('PyNumber_InPlaceAdd'),
        PyNumber_Subtract: exp('PyNumber_Subtract'),
        PyNumber_Multiply: exp('PyNumber_Multiply'),
        PyNumber_InPlaceMultiply: exp('PyNumber_InPlaceMultiply'),
        PyNumber_And: exp('PyNumber_And'),
        PyNumber_InPlaceAnd: exp('PyNumber_InPlaceAnd'),
        PyNumber_Or: exp('PyNumber_Or'),

```

```

PyNumber_InPlaceOr: exp('PyNumber_InPlaceOr'),
PyNumber_Xor: exp('PyNumber_Xor'),
PyNumber_InPlaceXor: exp('PyNumber_InPlaceXor'),
PyNumber_Lshift: exp('PyNumber_Lshift'),
PyNumber_Rshift: exp('PyNumber_Rshift'),
PyNumber_InPlaceRshift: exp('PyNumber_InPlaceRshift'),
PyNumber_Power: exp('PyNumber_Power'),
PyNumber_Remainder: exp('PyNumber_Remainder'),
PyNumber_Index: exp('PyNumber_Index'),
PyObject_RichCompare: exp('PyObject_RichCompare'),
PyObject_RichCompareBool: exp('PyObject_RichCompareBool'),
PyObject_CallFunctionObjArgs: exp('PyObject_CallFunctionObjArgs')
};

PyUnicode_AsUTF8 = tryCreateNativeFunction(symbols.PyUnicode_AsUTF8,
'pointer', ['pointer'], 'PyUnicode_AsUTF8');
PyObject_Repr = tryCreateNativeFunction(symbols.PyObject_Repr, 'pointer',
['pointer'], 'PyObject_Repr');
PyObject_Str = tryCreateNativeFunction(symbols.PyObject_Str, 'pointer',
['pointer'], 'PyObject_Str');
PyLong_Check = tryCreateNativeFunction(symbols.PyLong_Check, 'int',
['pointer'], 'PyLong_Check');
PyLong_AsLongLong = tryCreateNativeFunction(symbols.PyLong_AsLongLong,
'int64', ['pointer'], 'PyLong_AsLongLong');
PyLong_AsUnsignedLongLongMask =
tryCreateNativeFunction(symbols.PyLong_AsUnsignedLongLongMask, 'uint64',
['pointer'], 'PyLong_AsUnsignedLongLongMask');
PyLong_Type = symbols.PyLong_Type;
Py_DecRef = tryCreateNativeFunction(symbols.Py_DecRef, 'void', ['pointer'],
'Py_DecRef');
PyErr_Fetch = tryCreateNativeFunction(symbols.PyErr_Fetch, 'void',
['pointer', 'pointer', 'pointer'], 'PyErr_Fetch');
PyErr_Restore = tryCreateNativeFunction(symbols.PyErr_Restore, 'void',
['pointer', 'pointer', 'pointer'], 'PyErr_Restore');

hookLongConverters();
hookNumberOps();
hookCompareOps();
hookCythonInlineIntFactories();
hookCallFunctionObjArgs();

logLine(`\n[+] Hooks installed on ${pymod.name}`);
logLine('[+] PyNumber_* operations + Compare ops are now being traced in
HEX...\n');
logLine(`[+] TRACE_ONLY_DURING_PY_CALL=${TRACE_ONLY_DURING_PY_CALL}\n`);
logLine(`[+] TRACE_CYTHON_INLINE_INTS=${TRACE_CYTHON_INLINE_INTS}\n`);
}

function watchPythonLoad() {
  ['LoadLibraryW', 'LoadLibraryExW'].forEach(name => {
    const addr = findExportCompat('kernel32.dll', name);
    if (!addr) return;

    Interceptor.attach(addr, {
      onEnter(args) {
        this.lib = readUtf16Safe(args[0]);

```

```

    },
    onLeave(_) {
        const lower = String(this.lib || '').toLowerCase();
        if (lower.indexOf('python') !== -1) {
            logLine(`[+] ${name} loaded ${this.lib}`);
            setTimeout(installHooks, 0);
        }
    }
});
logLine(`[+] watching ${name}`);
});
}

installHooks();
watchPythonLoad();

```

用frida抓一下执行流，获取加密算法

$((0x5a+i*0xd)\&0xff)^{0x23} \wedge \text{input}[i] = \text{enc}[i]$

```

#!/usr/bin/env python3

FLAG_KEY = 0x23

ENCODED_FLAG = bytes(
    [
        0x14,
        0x2D,
        0x39,
        0xCB,
        0xE1,
        0xC3,
        0xF2,
        0xA6,
        0x94,
        0xB3,
        0xCB,
        0xB8,
        0xE6,
        0x7F,
        0x5E,
        0x0A,
        0x67,
        0x61,
        0x06,
        0x43,
        0x22,
        0x25,
        0x6F,
        0xD6,
        0xEE,
        0xD1,
        0xBB,
    ]
)

```

```

        0xE9,
        0x91,
        0xC3,
        0xB1,
        0x91,
        0xB1,
        0x4D,
        0x5F,
        0x6B,
        0x65,
        0x71,
        0x4A,
        0x57,
        0x60,
        0x31,
    ]
)

def mask_at(index: int) -> int:
    return ((0x5A + index * 13) & 0xFF) ^ FLAG_KEY

def decrypt() -> str:
    flag = []
    for index, encoded in enumerate(ENCODED_FLAG):
        plain = encoded ^ mask_at(index)
        if plain == 0:
            break
        flag.append(chr(plain))
    return "".join(flag)

if __name__ == "__main__":
    print(decrypt())

```

获取到flag:

```
miniL{y0u_4r3_m4nua1_m4p_m4st3r_hihihi!!!}
```